

MULTI-AGENT PATH FINDING · PROJECT REVIEW

SPAR

Forty years of multi-agent path finding, the warehouse problem it grew into, and what a directional, congestion-aware lifelong planner actually bought us.

5

PLANNERS COMPARED

183

TESTS IN THE SUITE

0

RUNTIME DEPENDENCIES

SPAR-PIBT — Lifelong Aisle-Managed Priority Inheritance with Backtracking. SPAR is the implementation, SPAR-PIBT is the algorithm.

1

SPEAKER NOTES

Set the frame before any detail: this is a review of a research implementation, and the honest headline is that the baseline we set out to beat wins on most maps. The deck earns that admission by showing the measurements rather than burying it.

The three numbers are deliberate. Five planners because the comparison is against published work, not against ourselves. Zero dependencies because every number here can be reproduced with `python3 main.py` on a clean machine.

WHAT THIS DECK COVERS

Three questions

1

Where did MAPF come from?
From decoupled prioritised planning in 1987, through optimal solvers, to the greedy planners that made thousands of robots tractable.

2

What problem were we solving?
Warehouse aisles are narrow and effectively one-way. PIBT is direction-blind. SPAR-PIBT adds an aisle layer that decides direction, and a recovery layer that survives it.

3

How, and did it work?
A two-layer architecture, six repaired mechanisms, and a hypothesis suite that scores each claim on the quantity it actually names. Three of six hold.

The design principle that runs through all three: direction, congestion and turning cost decide **which legal move PIBT tries first** — they never replace its collision checks or its backtracking.

2

SPEAKER NOTES

Flag the third bullet now rather than at the end. An audience told at the start that half the hypotheses failed listens to the evidence; an audience told at slide 30 listens for what else was hidden.

THE PROBLEM

Multi-agent path finding

Given a graph and k agents, each with a start and a goal, find k paths such that every agent arrives and no two agents ever collide.

Time is discrete. At each timestep an agent moves along one edge or waits. Everything else in MAPF — makespan, sum-of-costs, or the throughput objective this project uses — is an optimisation layered on top of that one rule.

Two kinds of collision define the problem. They are the next slide, and they are the **only** two things the low level of this system checks.

3

SPEAKER NOTES

Keep this slide short on purpose. The definition is not the interesting part; the two collision types on the next slide are, because the entire architectural argument later ("the high level never decides safety") is an argument about who is allowed to enforce exactly those two rules.

THE PROBLEM

Two collisions, and nothing else



both claim the shaded cell at $t+1$.

Vertex conflict

Two agents occupy the same cell at the same timestep.



legal for each agent alone, illegal for the pair

Swap conflict

Two agents exchange cells across one edge in one timestep.

4

SPEAKER NOTES

The swap conflict is the one people forget. Each agent's move is individually legal — each is moving into a cell the other is vacating — and only the pair is illegal. It is also the one that makes a naive "check the destination is empty" implementation wrong.

Worth saying out loud: these two checks are the only hard rules in our low level. Every later slide about the aisle layer is about resisting the temptation to add a third.

WHY IT RESISTED FORTY YEARS OF ATTACK · 1 OF 3

The joint action space is exponential in the agents

Each agent may move in four directions or wait. For k agents planned together, the joint branching factor is 5^k .

Coupled search is therefore exponential in the number of agents, not in the size of the map. Doubling the floor space is cheap; adding ten robots is not.

$\approx 5^k$

JOINT BRANCHING FACTOR

5

SPEAKER NOTES

Concretely: ten agents is about ten million joint actions per timestep. This is the number that pushed the field towards decoupling, and it is the number PIBT sidesteps by never planning more than one timestep ahead.

WHY IT RESISTED FORTY YEARS OF ATTACK · 2 OF 3

Optimal MAPF is NP-hard

NP-hard for both makespan and sum-of-costs (Yu & LaValle, 2013), anticipated by Ratner & Warmuth on the generalised 15-puzzle in 1986.

So optimality is not a matter of finding a cleverer algorithm. Any planner that scales is giving something up, and the interesting question is which thing.

NP-hard

TO SOLVE OPTIMALLY

6

SPEAKER NOTES

Do not linger. The point is only to establish that the tradeoff table two slides on is forced, not chosen.

WHY IT RESISTED FORTY YEARS OF ATTACK · 3 OF 3

In a warehouse, the horizon never ends

A delivered task is immediately replaced by a new one.
There is no final goal configuration to plan to.

The classical formulation does not get harder here. It stops applying. This is the turn that made lifelong MAPD a separate field, and it is the setting this project lives in.

Endless

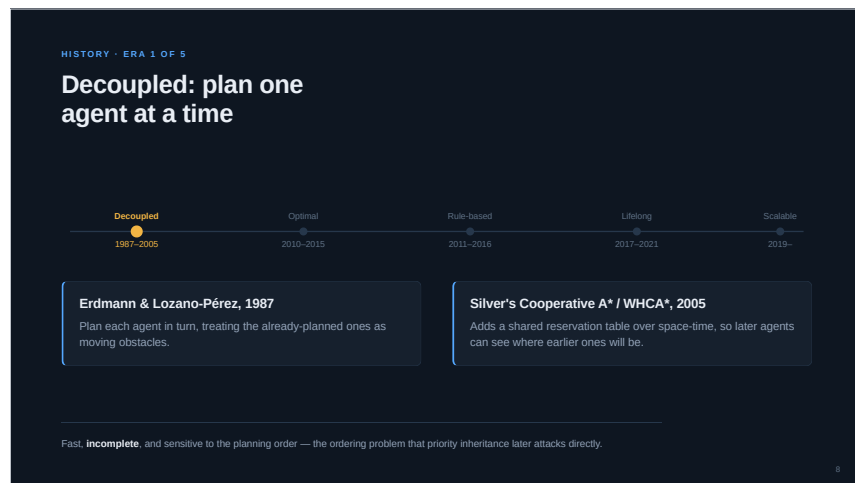
NO TERMINAL STATE

So the field split. One branch chased optimality and paid in scale. One gave up optimality for completeness. A third gave up both for speed — and that turned out to be the branch warehouses needed.

7

SPEAKER NOTES

This slide is the hinge of Act 1. Everything before it is classical MAPF; everything after it is about what you do when there is no plan to finish.



SPEAKER NOTES

The through-line to flag: this era's weakness is that the priority order is fixed in advance. PIBT's contribution, twenty years later, is letting the order change during the timestep.

HISTORY · ERA 2 OF 5

Optimal: search the conflicts, not the joint space

Decoupled

●

1987-2005

Optimal

●

2010-2015

Rule-based

●

2011-2016

Lifelong

●

2017-2021

Scalable

●

2019-

CBS — Sharon et al.

Find a conflict, split on it, and re-plan the two single agents under the resulting constraints. Optimal and genuinely elegant.

ICTS, M*, branch-and-cut-and-price

Different decompositions of the same idea, all following within a few years.

And it fails over past tens of agents in dense maps: the conflict tree grows with the conflicts, and a dense warehouse is nothing but conflicts.

9

SPEAKER NOTES

If asked why we did not benchmark against CBS: at the densities in this project it does not finish. The two baselines we do run — Token Passing and RHCR — are the lifelong ones, which is the honest comparison class.

HISTORY · ERA 3 OF 5

Rule-based: complete by construction

Decoupled

Optimal

Rule-based

Lifelong

Scalable

1987–2005

2010–2015

2011–2016

2017–2021

2019–

Push and Swap, Push and Rotate, TASS

A fixed repertoire of local manoeuvres that provably terminates in polynomial time.

The trade

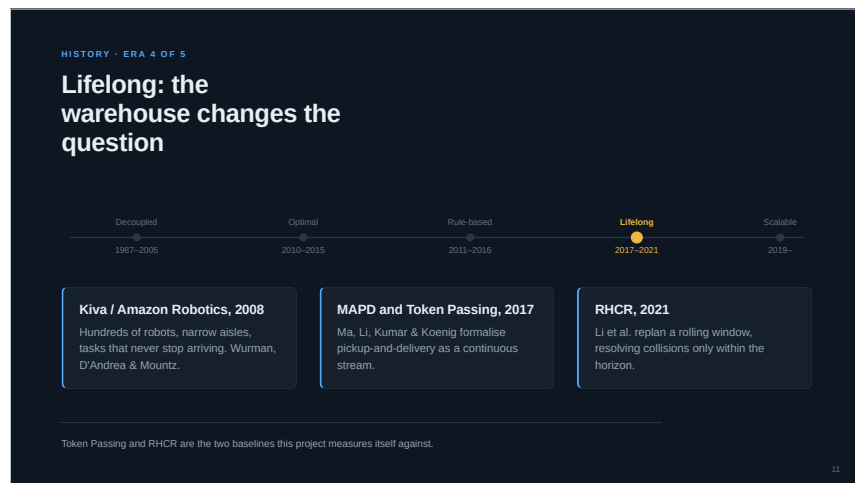
Solution quality is poor — but they never fail to terminate, which no decoupled planner can promise.

This era is where the seven-level recovery ladder later in this deck comes from: when a general planner is stuck, fall back on rules that always finish.

10

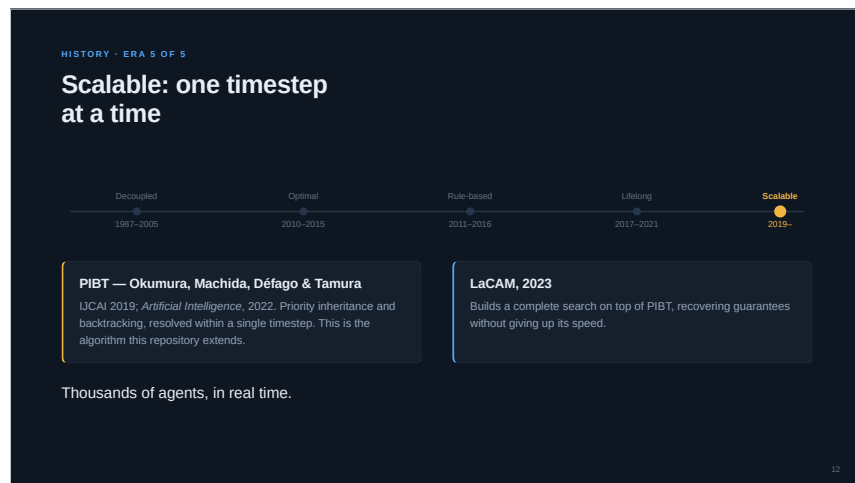
SPEAKER NOTES

Explicit link forward. Our recovery ladder is unashamedly rule-based, and it is in the system for exactly the reason this era existed.



SPEAKER NOTES

Commercial pressure, not theory, produced this era. Worth saying: the reframing was forced by a real floor, and that is also why the objective changed from makespan to throughput.



SPEAKER NOTES

PIBT is the whole basis of this project, so give it a beat. The key insight to plant here: it never plans more than one timestep, which is why the 5^k problem never arises.

HOW THE FAMILIES TRADE OFF

You choose two of three: optimal, complete, fast

FAMILY	REPRESENTATIVES	GUARANTEE	SCALE	THE PRICE
Optimal search	CBS, ICTS, M*, BCP	Optimal + complete	Tens of agents	Exponential in conflicts
Bounded suboptimal	ECBS, EECBS	Within a factor of optimal	Low hundreds	Focal search overhead
Rule-based	Push and Swap, Push and Rotate	Complete, polynomial	Hundreds	Poor solution quality
Greedy / reactive	PIBT, LaCAM, this repo	Conditional progress only	Thousands, real time	No global optimality

SPAR lives on the bottom row. The question the project asks: can a warehouse-aware layer on top of a greedy planner **buy back** some of the quality the greedy choice gave away?

13

SPEAKER NOTES

That closing question is the thesis of the whole project, so state it as a question and let the results answer it. The short version of the answer, for anyone who asks now: on corridor maps yes, on open grids no.

THE TURN THAT MADE THIS PROJECT NECESSARY

From one-shot MAPF to lifelong MAPD

Classical MAPF	Lifelong MAPD
k agents, k goals, fixed in advance.	Tasks arrive as a stream — Poisson, bursty or scheduled.
Plan once, offline, then execute.	A robot that delivers is immediately re-assigned.
Success is every agent parked on its goal.	There is no terminal configuration to plan to.
Objective: makespan or sum-of-costs.	Objective: throughput, service time, and its tail.
The plan is valid until it finishes.	Every plan is provisional; you replan forever.

14

SPEAKER NOTES

The line that matters most is the last one on the right. Because every plan is provisional, a mechanism that is expensive to compute has to justify itself every single timestep — which is the cost argument behind the 56–356× numbers later.

THE CORE WE BUILD ON

PIBT, in four steps

1

Candidates

The four neighbours of the current cell, plus staying put. Nothing longer than one step is ever planned.

2

Hard rejection

Off-graph, kinematics, vertex conflict, swap conflict. Nothing else — a rule that deletes a legal move breaks the progress argument.

3

Score and sort

Survivors are ranked by $S(v)$. Direction, congestion, turning cost and aisle rules all act here, and only here.

4

Inherit, then backtrack

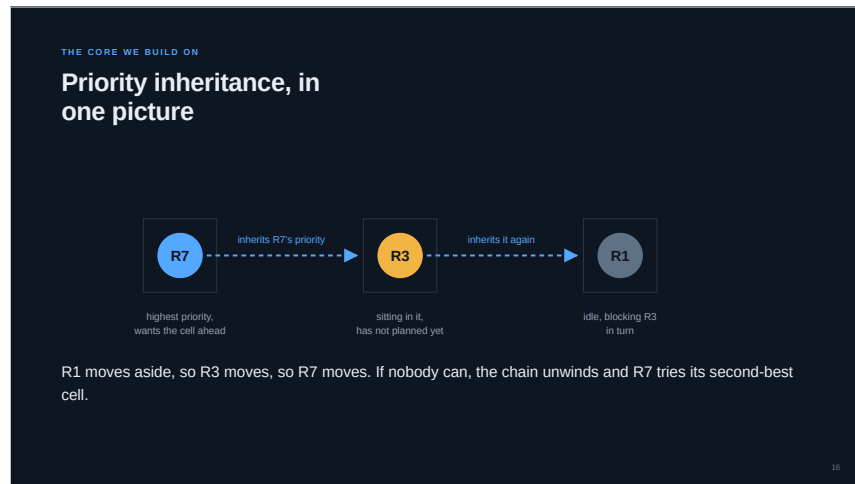
If the best cell holds an unplanned robot, that robot plans next with the caller's priority. If it fails, undo and try the next candidate.

Cost per timestep $O(|A|(\Delta(G) + F + \log|A|))$. Measured at **1.00 ms/step** for 40 robots on warehouse_medium, 1.90 with the aisle layer on.

15

SPEAKER NOTES

Step 2 is the one to emphasise, because the central defect found in this review was that our aisle layer had quietly added rules to it.



SPEAKER NOTES

This is the mechanism the whole deck turns on. The chain only works because R1 has *somewhere to go*. Delete one of R1's legal moves — which is exactly what a hard one-way rule does — and the chain dead-ends, R7 never moves, and the aisle layer that deleted the move gets blamed for the congestion it caused.

THE GAP WE SET OUT TO CLOSE · 1 OF 2

PIBT does not know what an aisle is

single-file corridor — no passing



each is moving optimally, right up to the moment neither can move at all

Direction-blind

PIBT scores a cell by how much closer it gets you. Nothing in that score knows the corridor has a flow.

No structural memory

Nothing records that this corridor was flowing north five steps ago, so the flow flaps with the traffic instead of settling.

17

SPEAKER NOTES

The picture does the work here. Both robots are behaving correctly by their own objective, which is what makes this a coordination failure rather than a bug.

THE GAP WE SET OUT TO CLOSE · 2 OF 2

And the matching is blind to it too

Task assignment picks whichever robot is nearest. But nearest *through a jammed corridor* is not nearest — and a greedy match keeps feeding robots into the jam that made it slow.

Two blind spots, one cause: no part of the system has a representation of the corridor as a shared resource with a state.

Robots generate directional requests, but aisles make directional decisions.

the architectural commitment of SPAR-PIBT, stated verbatim in the code

A robot may **prefer** a direction, the aisle **owns** it — and the aisle is the only thing that can see every robot that wants to use it.

18

SPEAKER NOTES

The quote is the design commitment, and every mechanism in Act 4 follows from it. If someone challenges the whole approach, this is the sentence to defend.

HOW IT IS BUILT

Two layers, and a line neither crosses

HIGH LEVEL — WHAT EACH ROBOT WANTS

task assignment waypoints routes directional demand aisle direction hysteresis + drain reservations priorities

deadlock detection and recovery

ranks candidates and prices the ones it dislikes — never removes them

↓
constrains + ranks candidates

LOW LEVEL — PIBT, ONE SYNCHRONISED TIMESTEP

candidates hard rejection score-sorted priority inheritance backtracking validate every step

the only place collision freedom is decided

Zero vertex and zero swap conflicts

across every map, density, seed and variant tested — checked every timestep by `validate_plan`.

SPEAKER NOTES

The invariant has two halves and people usually only hear the first. No high-level formula can produce a collision — and none can remove a legal move either. The second half is the one the code got wrong.

HOW, PART 1

One scoring function carries the whole high level

$$S_i(v) = \alpha \cdot \text{progress} + \beta_i \cdot [\text{preferred direction}] + \gamma_i \cdot [\text{stays in aisle}] \\ - \lambda \cdot \text{turn} - \mu \cdot \text{congestion} - v \cdot \text{wait} - \xi \cdot [\text{bottleneck}] - \zeta \cdot [\text{counterflow}]$$

 $\alpha = 10.0 \cdot \text{progress}$

Steps closer to the waypoint.
Dominates everything else by design.

 $\zeta = 8.0 \cdot \text{counterflow}$

The price of driving against a committed
direction. This is where the aisle layer
acts.

 $\lambda = 0.5 \cdot \text{turning}$

A flat penalty per direction change. The
cheapest term here, and the strongest
single mechanism in the system.

20

SPEAKER NOTES

Do not read the formula out. Point at α and ζ and say the ordering matters more than the values, then go to the next slide, which is about exactly that.

λ being both the cheapest term and the most effective mechanism is the single most surprising result in the project. It is worth a pause.

HOW, PART 1

The ordering is the design: $\alpha > \zeta > \beta > \gamma > \lambda$

A whole step of progress beats everything. Driving the wrong way down an aisle beats every other preference. A single turn is the cheapest thing you can be asked to pay.

 $\zeta < \alpha$, and ζ is a price

The aisle terms are penalties, not rejections. A robot drives the wrong way when that is the only way to make progress — and pays for it.

 $\mu \cdot C$ must be normalised

Unnormalised, the congestion mixture is a robot count and $\mu \cdot C$ reaches the scale of $\alpha \cdot \Delta$ — measured mean 3.40, p90 5.75, max 9.60. Congestion then outranks the terms it is meant to modulate.

Both conditions were false in the code until this review. The ordering the score claimed was not the ordering it had.

21

SPEAKER NOTES

This is the technical heart of the review. Two independent bugs, both of which made the same class of mistake: a mechanism meant to modulate was given enough weight to overrule.

HOW, PART 2

Aisles decide, and hold the decision



22

SPEAKER NOTES

The drain state is the non-obvious one. You cannot reverse a corridor that still has robots in it, so a flip is always a two-phase commit: empty it, then turn it.

HOW, PART 2

Two clocks, not one

Demand

Each robot contributes weighted demand for FORWARD or REVERSE in the aisle it is about to enter. The aisle sums both sides.

Minimum green

A committed direction is locked for $T_{\min}$ steps, and flips only outside a $\pm\tau$ dead band — so it cannot flap with the traffic.

Maximum green

Past $T_{\max}$, any opposing demand forces a flip. Without it a balanced aisle starves the side that never wins the imbalance.

Entry reservations

A robot at an entrance gets a ticket only if occupancy plus pending tickets stay under capacity. Tickets expire after 15 steps, so a stalled robot cannot hold a corridor hostage. Direction restricts *entry*, not interior movement — and it is priced, not enforced.

23

SPEAKER NOTES

Hysteresis alone is only half a traffic signal: it bounds how soon a direction may change, and nothing bounds how long it may persist. Pickups on one side of the map and deliveries on the other keeps demand permanently balanced, so without the maximum green an aisle holds one direction forever.

HOW, PART 3

A stall is only a stall when something corroborates it

In dense lifelong traffic a robot waits `t_blocked` steps as a matter of course. "No progress" is therefore a *precondition*, never a trigger.

The precondition

A group of robots has made no route progress for several timesteps.

The corroboration

A wait-for cycle in the dependency graph, or a repeated configuration in the position history. Only then does recovery escalate.

Worth **22 – 134 tasks per 1000 steps** on `warehouse_corridors`. Levels 5–7 are destructive when nothing is actually stuck.

24

SPEAKER NOTES

This effect was invisible before the other fixes landed, because the hard aisle constraints were manufacturing real deadlocks for recovery to rescue. Remove the cause and the false positives show through — a good example of why fixing bugs in order matters.

HOW, PART 3

Seven levels, one per timestep

- 1 Recompute routes from scratch
- 2 Unlock and re-decide the affected aisles
- 3 Release stale aisle reservations
- 4 Escalate the blocked robots' priorities
- 5 Allow temporary reverse movement
- 6 Send robots to escape vertices
- 7 Drop all directional constraints locally

One level per timestep, not all seven at once: in a synchronous loop you cannot observe whether a remedy worked until the next step. Firing all seven would apply six of them **blind**.

25

SPEAKER NOTES

The ladder is ordered by how much it gives up. Level 1 costs nothing but CPU; level 7 abandons the entire contribution of this project locally. That ordering is deliberate — you spend the cheap remedies first.

WHAT WE FOUND WHEN WE CHECKED · DEFECT 1 OF 4

The aisle layer broke our own invariant

Aisle direction and reservations were implemented as **hard rejections**, deleting legal moves before scoring — while the architecture slide says the high level never replaces PIBT's backtracking.

Priority inheritance needs to be able to push a robot aside. A rejected move is a move it can no longer use to unblock a chain.

1.9–3.1×

THROUGHPUT, PRICING INSTEAD OF REJECTING

Measured on corridors 84 → 158, narrow 112 → 304, medium 161 → 405 tasks per 1000 steps: **p = 0.008** on each.

26

SPEAKER NOTES

This is the largest single effect in the repository, and it came from deleting code rather than adding any. Say that plainly — it is the most useful lesson in the deck.

WHAT WE FOUND WHEN WE CHECKED · DEFECT 2 OF 4

Two hypotheses were never actually tested

H3's treatment cell was empty

The reservation layer was gated on `direction_control == "aisle"`, and the variant built to isolate it sets `"robot"`. The treatment was a bit-identical copy of the control, so its main effect was necessarily zero.

And on one map it never fires

On bottleneck, no aisle ever commits a direction at all — so the mechanism under test was never active.

"Exactly zero effect" described **the wiring**, not the mechanism. A result that clean should have been suspicious on its own.

27

SPEAKER NOTES

The methodological point: an effect size of exactly zero, to full precision, is nearly always a bug. Real mechanisms are noisy even when they do nothing.

WHAT WE FOUND WHEN WE CHECKED · DEFECT 3 OF 4

The aisle signal could starve

A committed direction was held indefinitely unless the imbalance crossed τ . But pickups on one side and deliveries on the other keeps demand permanently *balanced*.

So the imbalance never crossed the threshold, the direction never flipped, and the traffic wanting the other way never moved.

35 / 35

ROBOTS STALLED BY $T = 120$, CORRIDORS

Fixed by the maximum green: past T_{max} , any opposing demand at all forces a drain and a flip.

28

SPEAKER NOTES

Note the interaction with defect 1: the maximum green was decisive while direction was a hard constraint, because there was no other way out. Once counterflow is merely expensive, a robot can buy its way out, so the two fixes are partly redundant. It stays because it makes the layer correct, not merely survivable.

WHAT WE FOUND WHEN WE CHECKED · DEFECTS 4-6

Three more, each measured

Bent aisles

Aisles were being segmented into L and U shapes, so one-waying a corner blocked it in both directions.

Recovery on healthy traffic

Escalation fired on ordinary queueing, applying destructive remedies to a system that was working.

Congestion outranked its own targets

The unnormalised mixture let μ -C dominate the terms it was meant to modulate.

183 tests, up from 161. Task assignment is held identical across all five planners, so the baseline comparison isolates the movement layer and nothing else.

29

SPEAKER NOTES

Each of these has a regression test now. The test count going up by twenty-two is the durable part of this review; the throughput numbers will move again, the tests will keep the mechanisms honest.

RESULT 1 · AGAINST THE LITERATURE

Plain PIBT wins, and not narrowly

lifelong_pibt beats both external baselines on every map, $p < 0.001$ on throughput in every cell. Token Passing completes **literally zero** tasks on warehouse_corridors across all ten seeds.

Why: no inheritance

All 16 robots queue nose-to-tail in the connecting corridor and never move again. A queue Token Passing forms, it cannot resolve.

And it is 56–356× cheaper

lifelong_pibt 1.01 ms/step

full_lda_pibt 4.06

RHCR 85.4

token_passing 318.4

10 seeds × 400 timesteps, permutation test vs lifelong_pibt. Assignment is held constant, so neither baseline runs its own paper's policy and RHCR's window is an untuned default.

30

SPEAKER NOTES

Do not oversell this. The caveat in the footer is real: we hold assignment constant to isolate the movement layer, which means neither baseline is running the full system its authors published. It is the right experiment for our question and the wrong one for "is RHCR good".

RESULT 1 · WHERE SPAR-PIBT LANDS

The aisle layer wins where corridors are the constraint

bottleneck full_lda_pibt now wins: 145 vs 118 tasks per 1000 steps, $p = 0.016$.	corridors Ties, where it used to lose 20 to 160 tasks per 1000 steps.	medium and narrow Still loses to plain PIBT. Turning cost alone beats the full aisle layer on both.
---	---	---

The honest summary: the aisle layer earns its cost exactly where the map makes direction a real constraint, and costs more than it returns everywhere else.

31

SPEAKER NOTES

This slide is the answer to the thesis question posed on slide 13. Say it as a finding, not an apology: a mechanism with a map-dependent payoff is a useful result, provided you can say which maps.

RESULT 2 · DE-CONFOUNDING

With both cells live, the attribution reverses

On warehouse_medium, once H3's treatment cell actually differed from its control:

Aisle direction alone 337 → 405	Entry admission alone 337 → 262	Both together 217
------------------------------------	------------------------------------	----------------------

Two mechanisms that were reported as one. Separated, one helps and one hurts — and together they are worse than either.

32

SPEAKER NOTES

The interaction is the interesting part: both together is worse than admission alone. That suggests entry admission is not merely useless but actively fights the direction signal, which is the first item on the future-work slide.

RESULT 2 · DE-CONFOUNDING

One flag, three mechanisms

congestion_aware moved the movement score, the assignment cost and the blocking term *together*.

So H4 — “congestion in matching cuts service time” — was never a test of congestion in matching. It was a test of all three at once, and the same confound the factorial designs were built to remove, one level further down.

The general lesson: an ablation flag that controls more than one mechanism is not an ablation. It is a **bundle**, and its effect size cannot be attributed to anything.

33

SPEAKER NOTES

Keep this slide sparse. It is a methodology point and it lands better as one sentence than as a table.

RESULT 3 · THE SCOREBOARD

Each claim, scored on its own metric

	CLAIM	VERDICT
H1	Aisle direction lifts per-aisle flow in narrow aisles	not on its own metric
H2	Hysteresis reduces switching and prevents oscillation	supported, every map
H3	Entry capacity admission cuts head-on conflicts	map-dependent
H4	Congestion in matching cuts service time	not supported
H5	Proximity decay of the direction weight helps arrival	mechanism is inert
H6	Localised recovery beats a global planner	supported on corridors

Scored on the quantity each claim **names** — head-on conflicts for H3, per-aisle flow for H1 — not on throughput. H1 fails its own metric yet raises end-to-end throughput on three of four maps.

34

SPEAKER NOTES

The footer is the point of the whole slide. Scoring every hypothesis on throughput would have marked H1 as a success and told us nothing about why. Scoring it on per-aisle flow tells us the mechanism does not do what we said it does, even though the system is better with it.

WHAT THE FIXES WERE WORTH

Three mechanisms, measured one at a time

1 Direction ranks candidates instead of rejecting them

The largest effect in the repository: 84 → 158 on corridors, 112 → 304 on narrow, 161 → 405 on medium, $p = 0.008$ on each.

2 Recovery escalates only on a corroborated stall

22 → 134 tasks per 1000 steps on warehouse_corridors — invisible before, because the hard constraints were manufacturing the deadlocks recovery was rescuing.

3 The maximum green, and why it now matters less

Decisive while direction was a hard constraint. Once counterflow is merely expensive a robot can buy its way out, so the two fixes are partly redundant. It stays because it makes the layer correct, not just survivable.

Coordinated direction assignment across parallel aisles — the last review's most promising fix — is implemented, and measures **±11 tasks per 1000 steps**.

35

SPEAKER NOTES

The footer deserves a sentence out loud: the fix we expected most from did nothing measurable, and the fix that mattered most was removing code. It is worth being explicit that our predictions were poor.

WHERE THIS GOES

Next, in order of expected payoff

1

Make entry admission earn its cost
The one repaired mechanism that still loses throughput on every map. Both the capacity model and the admission rule are suspect.

2

Work out why the aisle layer loses on open grids
It wins on corridors and loses on medium and narrow. Turning cost alone beats it on two maps.

3

Tune the baselines honestly, then re-run
Pair each baseline with the assignment policy from its own paper, and size RHCR's window for its intended scale.

4

Close the gaps the spec still leaves open
Intersection reservations, kinematics and footprints, asynchronous execution, and the delayed/failed-robot scenarios.

36

SPEAKER NOTES

Item 3 is the one a reviewer will press on, and it should be conceded early: our baseline numbers answer "which movement layer is better", not "which published system is better".

IN CLOSING

Three things worth taking away

1 The core was the contribution all along

Forty years of MAPF converged on a mechanism — priority inheritance with backtracking — that a hundred lines of recursion capture. Plain lifelong PIBT still wins on the open grids.

2 A mechanism that cannot act is not a negative result

Three of six hypotheses were reported as failures. One had an empty treatment cell, one a map where the treatment never fired, one was rescuing deadlocks we had manufactured.

3 The invariant was load-bearing, not decorative

"Ranks candidates, never decides safety" appears three times in this deck. The code did the opposite, and that one line accounts for every collapse we blamed on aisle management.

`python3 main.py` — no install, no dependencies, 183 tests, zero collisions.

37

SPEAKER NOTES

End on the second point if there is time for only one. The most transferable lesson from this project is that a failed hypothesis deserves the same scrutiny as a successful one, because the most common cause of a clean null result is a treatment that never ran.